

Assignment 1: Enterprise Knowledge Copilot (Internal Chatbot)

Problem Statement: Create a POC to Develop chatbot for an organization which answers questions related to HR polies like leaves, internal employee related data.

Purpose:

This take-home assignment is designed to evaluate AI Fullstack Developer capability in building a production-oriented, end-to-end AI application. The focus is on frontend engineering excellence, API orchestration, LLM integration, application architecture, and software engineering.

You are expected to demonstrate how AI outputs are integrated into real user workflows with strong UX, reliability, and accurate considerations.

Quick Facts:

Role level	AI Fullstack Developer
Submission format	Source code repository + written materials + reproducible instructions
Primary dataset	Pick any of the public datasets from Kaggle or Github or generate synthesized data.
LLM	Use any LLM

General Submission Rules

- Use only public, synthetic, or clearly de-identified datasets. Do not use protected health information (PHI) or any private/company-confidential data.
- Your solution must run from a clean environment using the commands you document in README.
- Pin or otherwise clearly control dependencies and random seeds where appropriate.
- You may use Python and any common open-source library. Do not require paid tools or paid cloud services.
- You may use AI-assisted coding tools if you wish, but you must be able to explain and defend every design choice, implementation detail, and debugging step during the live interview.
- Include a short design rationale, assumption log, and “what I would improve with more time” note in your submission.
- Include meaningful commit history that reflects how the solution evolved; avoid submitting a single final dump of code.

Assignment Objective

- This assignment evaluates frontend AI engineering capability, including prompt design, tool/function orchestration, backend integration (Node.js or FastAPI), and user-centric AI workflow implementation.
- Because the role operates in an enterprise environment, you must also include a lightweight but concrete responsible-AI and governance approach covering explainability, feedback handling, and accuracy.

Problem Statement

- Using one publicly available document dataset (for example, GitLab Handbook, or an equivalent open-source knowledge base), build an **Enterprise Knowledge Copilot** application.
- You must implement a chat-based AI system that retrieves relevant information from the selected dataset and generates grounded responses using a public LLM. The solution should demonstrate **prompt design, backend orchestration, tool or retrieval integration, and Frontend user experience**.
- You do not need access to paid APIs or production infrastructure, but you must clearly explain how your design would scale to larger datasets, more users, and stricter enterprise governance requirements.

What You Must Build

- A React + TypeScript chat application with conversation history, loading/error states, and basic accessibility support.
- A lightweight backend API to handle chat requests, manage context, and orchestrate LLM calls with clear request/response contracts.
- Public LLM integration with handling for latency, failures, and basic conversation context.
- A short accuracy and limitations note, plus documentation of any optional features such as explainability UX or feedback controls.
- A lightweight backend service (Node.js or FastAPI) to orchestrate LLM calls, tools, and application logic.
- A short responsible-AI and governance note describing intended use, limitations, accuracy considerations, and user-feedback handling.

Build a web-based internal chatbot that helps employees ask questions and receive answers from a small, public knowledge base.

Example user questions:

- “Summarize our leave policy.”
- “What is the escalation process for incidents?”
- “Find information about onboarding steps.”

The goal is to build smooth flow from frontend to backend with UX, and AI integration.

Required Deliverables

- Frontend and backend source code for the Enterprise Knowledge Copilot application.
- A short note describing prompt design, tool/function usage (if required), and any AI interaction improvements implemented.
- Source code repository
- README with environment setup instructions, application run commands, and configuration steps for the LLM and dataset.
 - Environment setup
 - How to run frontend and backend
 - How to configure API keys
- Architecture overview (diagram or text)
- Accuracy & limitations
- A brief discussion of risks such as incorrect answers, over-reliance on AI, or UX limitations relevant to the chosen dataset.

Constraints

- The application must be runnable on standard developer hardware using free, publicly available tools; no paid services or special infrastructure are required.
- Do not optimize only for UI polish or LLM output quality. Application correctness, reliability, UX clarity, and engineering maturity matter more than perfect AI responses.
- Use only public datasets, documents, LLM models, and libraries.
- Any assumptions, limitations must be clearly documented.

Submission Checklist

- Application setup and run instructions documented in the README.
- Frontend and backend integration is clearly documented.
- LLM configuration, prompt flow, and dataset usage documented.
- At least one AI interaction improvement or control documented (accuracy improvement or tool usage).
- Architecture and scale-out considerations documented (for example handling more users, larger datasets, or additional tools)